

Follow-up: Grammatiken

- **Frage:**

Was ist, wenn bei einer Wortgenerierung ein Wort mit Variablen entsteht, das nicht weiter zu einem Wort mit ausschließlich terminalen Symbolen verändert werden kann?

- **Antwort:**

Schauen wir uns noch einmal die Definition einer Sprache $L(G)$ an, die durch die Grammatik G erzeugt wird.

- Die von einer Grammatik $G = (V, \Sigma, P, S)$ **erzeugte Sprache** ist

$$L(G) = \{w \in \Sigma^* \mid S \Rightarrow_G^* w\}$$

- Menge aller Wörter aus Alphabetsymbolen, die aus der Startvariable S ableitbar sind.
- Wörter, die Variablen enthalten, gehören nicht zur Sprache, auch wenn sie nicht weiter verändert werden können.

Deterministische Turingmaschine

Total rekursive (berechenbare) Funktionen, entscheidbare und semi-entscheidbare Sprachen

- Eine Funktion $f: \Sigma^* \rightarrow \Sigma^*$ heißt **total rekursiv (berechenbar)**, wenn es eine TM gibt, die aus der Eingabe x den Funktionswert $f(x)$ berechnet.
- Eine Funktion $f: \mathbb{N}^k \rightarrow \mathbb{N}$ heißt **total rekursiv (berechenbar)**, wenn es eine TM gibt, die aus Eingaben vom Typ $bin(i_1), \dots, bin(i_k)$ die Ausgabe $bin(f(i_1, \dots, i_k))$ berechnet.
- Eine Sprache $L \subseteq \Sigma^*$ heißt **rekursiv (entscheidbar)**, wenn es eine TM gibt, die auf allen Eingaben stoppt und die Eingabe w genau dann akzeptiert, wenn $w \in L$ ist.
- Eine Sprache $L \subseteq \Sigma^*$ heißt **rekursiv aufzählbar (semi-entscheidbar)**, wenn es eine TM gibt, die genau diejenigen Eingaben w akzeptiert, die aus L sind.

Programmierung der TM am Beispiel

- Wir entwickeln eine TM für die Sprache $L = \{0^n 1^n \mid n \geq 1\}$
- Die Sprache L wird entschieden (wir sagen auch erkannt) durch die TM $(Q, \Sigma, \Gamma, B, q_0, \delta, F)$ mit
 - $Q = \{q_0, \dots, q_7\}$
 - $\Sigma = \{0, 1\}$
 - $\Gamma = \{0, 1, B\}$
 - $F = \{q_7\}$
 - δ gemäß Tabellen (später)
- TM arbeitet in zwei Phasen:
 - Phase 1: Teste, ob das Eingabewort von der Form $0^i 1^j$ für $i, j \geq 1$.
 - Phase 1 verwendet q_0 und q_1 und wechselt bei Erfolg nach q_2
 - Phase 2: Test, ob $i = j$ gilt.
 - Phase 2 verwendet q_2 bis q_6 und wechselt bei Erfolg nach q_7 .

Programmierung der TM am Beispiel

- q_0 :
 - Laufe von links nach rechts über die Eingabe bis ein Zeichen ungleich 0 gefunden wird.
 - Falls dieses Zeichen eine 1 ist, gehe über in Zustand q_1 .
 - Sonst ist dieses Zeichen ein Blank. Verwirf die Eingabe.
- q_1 :
 - Gehe weiter nach rechts bis zum ersten Zeichen ungleich 1.
 - Falls dieses Zeichen eine 0 ist, verwirf die Eingabe.
 - Sonst ist das gefundene Zeichen ein Blank. Bewege den Kopf um eine Position nach links auf die letzte gelesene 1. Wechsel in den Zustand q_2 , Phase 2 beginnt.

δ	0	1	B
q_0	$(q_0, 0, R)$	$(q_1, 1, R)$	-
q_1	-	$(q_1, 1, R)$	(q_2, B, L)

Programmierung der TM am Beispiel

- q_2 : Kopf steht auf dem letzten Nichtblank. Falls dieses Zeichen eine 1 ist, so lösche es, gehe nach links, und wechsle in den Zustand q_3 . Sonst verwirf die Eingabe.
- q_3 : Bewege den Kopf auf das erste Nichtblank. Dann q_4 .
- q_4 : Falls das gelesene Zeichen eine 0 ist, ersetze es durch ein Blank und gehe nach q_5 , sonst verwirf die Eingabe.
- q_5 : Wir haben jetzt die linkste 0 und die rechteste 1 gelöscht. Falls Restwort leer, dann q_7 (akzeptiere), sonst q_6 .
- q_6 : Laufe wieder zum letzten Nichtblank und starte erneut in q_2 .

δ	0	1	B
q_2	-	(q_3, B, L)	-
q_3	$(q_3, 0, L)$	$(q_3, 1, L)$	(q_4, B, R)
q_4	(q_5, B, R)	-	-
q_5	$(q_6, 0, R)$	$(q_6, 1, R)$	(q_7, B, R)
q_6	$(q_6, 0, R)$	$(q_6, 1, R)$	(q_2, B, L)

Konfigurationen und (direkte) Nachfolgekongfigurationen

- Eine **Konfiguration** einer TM ist ein String $\alpha q \beta$, für $q \in Q$ und $\alpha, \beta \in \Gamma^*$. Bedeutung: auf dem Band steht $\alpha \beta$ eingerahmt von Blanks, der Zustand ist q , und der Kopf steht unter dem ersten Zeichen von β .
- $\alpha' q' \beta'$ ist **direkte Nachfolgekongfiguration** von $\alpha q \beta$, falls $\alpha' q' \beta'$ in einem Rechenschritt aus $\alpha q \beta$ entsteht. Wir schreiben $\alpha q \beta \vdash \alpha' q' \beta'$
- $\alpha'' q'' \beta''$ ist **Nachfolgekongfiguration** von $\alpha q \beta$, falls $\alpha'' q'' \beta''$ in endlich vielen Rechenschritten aus $\alpha q \beta$ entsteht. Wir schreiben $\alpha q \beta \vdash^* \alpha'' q'' \beta''$. Insbesondere gilt $\alpha q \beta \vdash^* \alpha q \beta$

Beispiel zum Umgang mit Konfigurationen

- Die im obigen Beispiel beschriebene TM liefert auf der Eingabe 0011 die folgende Konfigurationsfolge
- Phase 1: $q_0 0011 \vdash 0q_0 011 \vdash 00q_0 11 \vdash 001q_1 1 \vdash 0011q_1 B \vdash 001q_2 1$
- Phase 2: Übung
- Beobachtung: abgesehen von Blanks am Anfang und Ende des Strings sind die Konfigurationsbeschreibungen eindeutig.

Techniken zur Programmierung von TMs

Techniken zur Programmierung von TMs

- Trick 1: Speicher im Zustandsraum
 - Für $k \in \mathbb{N}$ können wir k Zeichen des Bandalphabets im Zustand speichern
 - Vergrößere den Zustandsraum um den Faktor $|\Gamma|^k$, d.h. setze $Q_{neu} := Q \times \Gamma^k$.
- Vorsicht: funktioniert nur für konstantes k
 - d.h. k darf nicht von der Eingabe abhängen.
 - Wir können beispielsweise keinen Zähler, der die Länge der Eingabe zählt, im Zustandsraum realisieren.
 - Derartige Zähler müssen auf dem Band gehalten werden.

Techniken zur Programmierung von TMs

- Trick 2: **Mehrspurmaschinen**

- Bei einer k-spurigen TM handelt es sich um eine TM, bei der das Band in k sogenannte Spuren eingeteilt ist, d.h. in jeder Bandzelle stehen k Zeichen, die der Kopf gleichzeitig einlesen kann. Das können wir erreichen, indem wir das Bandalphabet um k-dimensionale „Vektoren“ erweitern, z.B. $\Gamma_{neu} := \Gamma^k$.
- Die Verwendung einer mehrspurigen TM erlaubt es häufig, Algorithmen einfacher zu beschreiben.

Techniken zur Programmierung von TMs

- Bsp: Addition. Aus der Eingabe $\text{bin}(i)\#\text{bin}(j)$ für $i, j \in \mathbb{N}$ soll $\text{bin}(i + j)$ berechnet werden. Verwende 3-Spurige TM mit

$$\Sigma = \{0, 1, \#\} \text{ und } \Gamma = \{0, 1, \#, \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}, \dots, \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}, B\}$$

Techniken zur Programmierung von TMs

- Schritt 1: Transformation in Spurendarstellung: Schiebe die Eingabe so zusammen, dass die Binärkodierungen von i und j in der ersten und zweiten Spur rechtsbündig übereinander stehen. Aus der Eingabe

0011#0110 wird beispielsweise $B^* \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} B^*$

Techniken zur Programmierung von TMs

- Schritt 2: Addition nach der Schulmethode, indem der Kopf das Band von rechts nach links durchläuft. Überträge werden im Zustand gespeichert. Als Ergebnis auf Spur 3 ergibt sich

$$B^* \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} B^*$$

- Schritt 3: Rücktransformation von Spur 3 ins Einspur-Format: Ausgabe 1001.

Techniken zur Programmierung von TMs

- **Trick 3: k-Band TM**

- Eine k-Band-TM ist eine Verallgemeinerung der Turingmaschine und verfügt über k Arbeitsbänder mit jeweils einem unabhängigen Kopf.

Die Zustandsübergangsfunktion ist entsprechend von der Form $\delta: Q \times \Gamma^k \rightarrow Q \times \Gamma^k \times \{L, R, N\}^k$.

Band 1 fungiert als Ein-/Ausgabeband wie bei der (1-Band) TM.

Die Bänder 2, .., k sind initial mit B^* beschrieben.

- **Bemerkung:** Bei der zuvor besprochenen mehrspurigen TM handelte es sich lediglich um eine hilfreiche Interpretation der (1-Band) TM. Die k-Band TM ist hingegen eine echte Verallgemeinerung. Der wesentliche Unterschied liegt darin, dass die Lese-/Schreib-Köpfe der einzelnen Bänder sich unabhängig voneinander bewegen können.

Techniken zur Programmierung von TMs

Satz: Eine k -Band TM M , die mit Rechenzeit $t(n)$ und $s(n)$ Platz auskommt, kann von einer (1-Band) TM M' mit Zeitbedarf $O(t^2(n))$ und Platzbedarf $O(s(n))$ simuliert werden.

Beweis (mit Mehrspurmaschinen)

Techniken zur Programmierung von TMs

Beweis (1)

- Die TM M' verwendet $2k + 1$ Spuren. Nach Simulation des t -ten Schrittes für $0 \leq t \leq t(n)$ gilt
 - Die ungeraden Spuren $1, 3, \dots, 2k-1$ enthalten den Inhalt der Bänder $1, \dots, k$ von M .
 - Auf den geraden Spuren $2, 4, \dots, 2k$ sind die Kopfposition auf diesen Bändern mit dem Zeichen # markiert.
 - Auf der letzten Spur stehen zwei Markierungen * und \$, wobei * die linke und \$ die rechte bis jetzt von einem Kopf erreichte Position markiert.
 - Ausnahme: Zu Beginn stehe \$ in Zelle 2, um Überlagerungen zu vermeiden.
- Diese Initialisierung der Spuren ist in Zeit $O(1)$ möglich.

Techniken zur Programmierung von TMs

Beweis (2) simulierte 2-Band-TM M

Diagram illustrating memory buffers for two different data sets:

- Top buffer: a r **b** e i t s b a n d 1
- Bottom buffer: a r b e i **t** s b a n d 2

simulierende 5-spurige TM M' (zu Beginn eines Simulationsschrittes)

	a	r	b	e	i	t	s	b	a	n	d	1							
			#																
						a	r	b	e	i	t	s	b	a	n	d	2		
										#									
	*														\$				

Techniken zur Programmierung von TMs

Beweis (3):

- Jeder Rechenschritt von M wird durch M' wie folgt simuliert.
 - Am Anfang stehe der Kopf von M' auf dem $*$ und M' kenne den Zustand von M .
 - Der Kopf von M' läuft nach rechts bis zum $\$$, wobei die k Zeichen an den mit $\#$ markierten Spurpositionen im Zustand abgespeichert werden.
Dafür muss der Zustand um $\dots \times \Gamma^k$ erweitert werden.
 - Am $\$$ -Zeichen angekommen kann M' die Übergangsfunktion von M auswerten und kennt den neuen Zustand von M sowie die erforderlichen Übergänge auf den k Bändern.
 - Nun läuft der Kopf von M' zurück, verändert dabei die Bandinschriften an den mit $\#$ markierten Stellen und verschiebt, falls erforderlich, auch die $\#$ -Markierungen um eine Position nach links oder rechts.
 - Falls nötig, werden während der Simulation des Schrittes von M auch die Markierungen $*$ und $\$$ um eine Position nach außen bewegt.

Techniken zur Programmierung von TMs

Beweis (4):

- Laufzeitanalyse:
 - Wieviele Bandpositionen können zwischen den Markierungen * und \$ liegen?
 - Nach t Schritten $t \geq 1$ können diese Markierungen höchstens $2t$ Positionen auseinanderliegen, da jedes der beiden Zeichen pro Schritt höchstens um eine Position nach außen wandert.
 - Also ist der Abstand zwischen diesen Zeichen und somit auch die Laufzeit zur Simulation eines Schrittes durch $O(t(n))$ beschränkt.
 - Insgesamt ergibt das zur Simulation von $t(n)$ Schritten eine Laufzeitschranke von $O(t^2(n))$.

QED

Techniken zur Programmierung von TMs

- Standardtechniken aus der Programmierung können auch auf TMs implementiert werden.
 - Schleifen haben wir bereits im Beispiel $L = \{0^n 1^n | n \geq 1\}$ gesehen.
 - Variablen können realisiert werden, indem wir Speicherbereiche auf einer Spur des Bandes reservieren. Der Name der Variablen sowie der Beginn und Ende des Speicherbereichs können auf einer weiteren Spur vermerkt werden. Falls der reservierte Speicherbereich nicht ausreicht, müssen ggf. Speicherbereiche umkopiert werden.
 - Unterprogramme können implementiert werden indem wir einzelne Zustände oder eine Spur des Bandes zur Realisierung eines Prozedurstacks reservieren. Der Stack ermöglicht auch dynamische Aufrufe von Unterprogrammen.
 - Parallele Rechnungen können auf verschiedenen Bändern durchgeführt werden. Dies ist offensichtlich für eine konstante Anzahl an parallel laufenden Rechnungen. Man kann sich aber überlegen (analog zur Simulationen mehrerer Bänder), dass dies für beliebig viele parallele Rechnungen möglich ist.
- Basierend auf diesen Techniken können wir auch auf bekannte Algorithmen, z.B. zum Sortieren von Daten zurückgreifen, aus „Algorithmen und Datenstrukturen“.